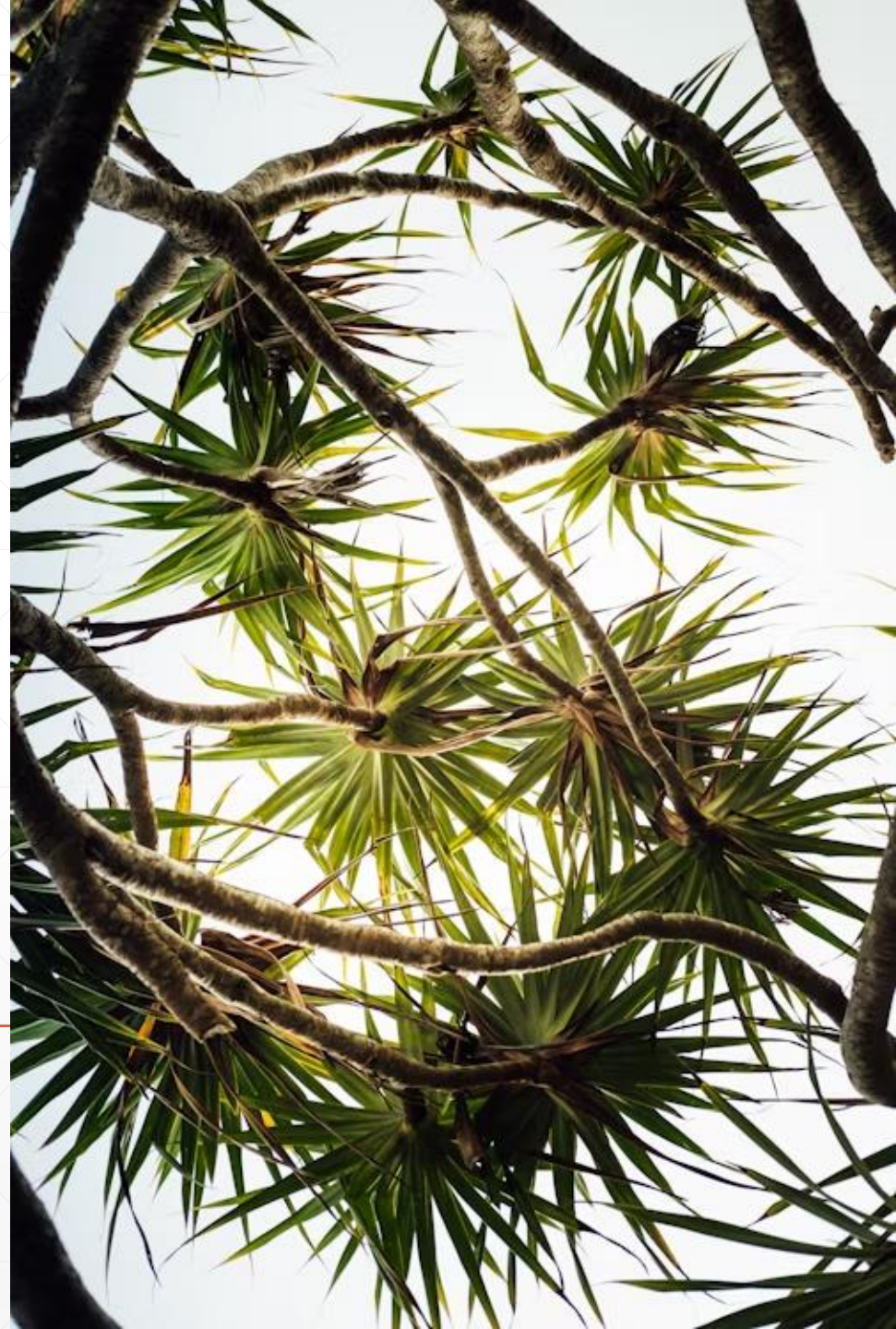
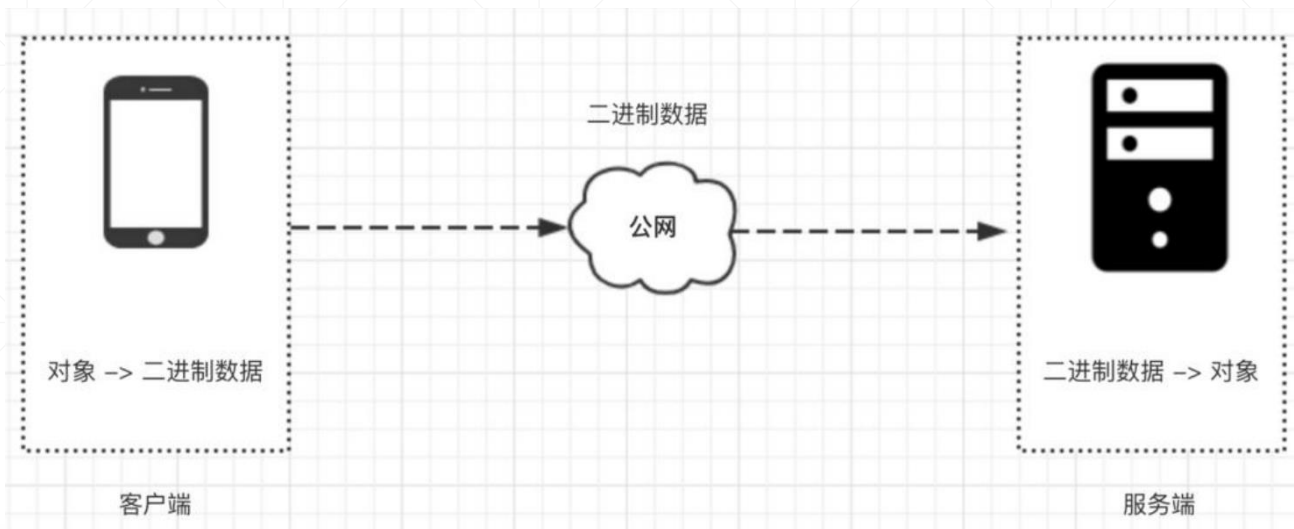


网络协议的设计与实现（上）

北京理工大学计算机学院
金旭亮



数据在网络中的传输



在网络中，数据是以二进制方式传送的，
可以看成是一个字节流。

但在我们编写的网络应用中，直接处理
二进制数据是很麻烦的，通常都采用
“对象”这种方式，实现各种业务逻辑。

这就要求必须有一个转换过程，完成对象与二进制数据之间的相互转换。与此同时，出于网络应用之间传输数据的要求，可能还要求在业务数据之外，附加一些包容特定信息的数据，比如“版本号”，这就要求我们在编写网络应用时，需要定制在客户端与服务端之间发送数据的具体形式，为此，需要针对特定的应用场景，确定相应的“网络通讯协议”。

“网络通讯协议”，规范了计算机之间交换信息的格式与流程。

依据特定场景自定义通讯协议，并且利用Netty内置的编码器和解码器机制实现，是需要重点学习与掌握的内容。

网络私有协议数据包结构



网络协议，通常拥有以下的数据包结构：

数据包头（包容多个字段，每个字段有特殊的含义）

数据主体（其数据的存取，需要由数据包头进行控制）

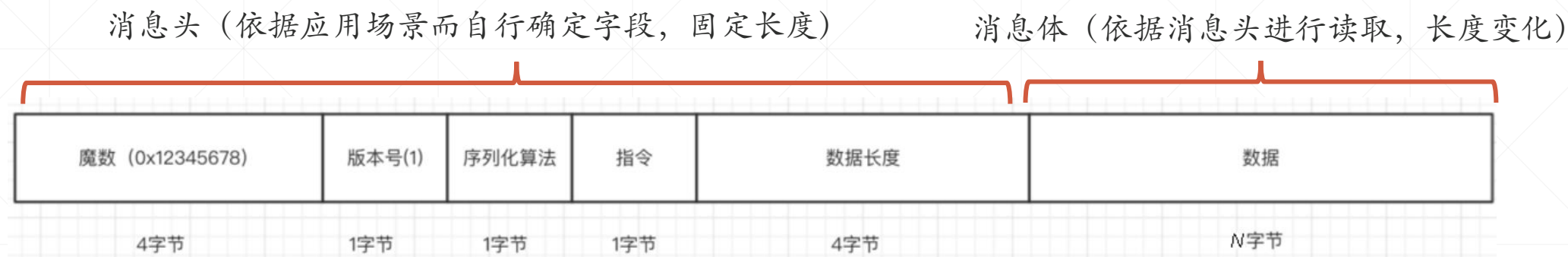


使用Netty实现时，数据是放到ByteBuf中的，需要编写一个ChannelHandler（多半从ByteToMessageDecoder派生），在它的decode()方法中按照私有协议的方式读取数据）



在读取数据时，要注意粘包和半包问题，因此，需要检查是否已经收到了完整的信息，如果没收到的，则应该放弃后继处理，等数据到来之后，再处理。Netty提供了现成的用于“消息定界”的Handler，可以直接重用。

自定义通信协议的示例



上述数据包中的“魔数”，是一个事先约定好的数字，它用于区分收到的数据是不是采用特定的自定义通信协议编码的。



序列化算法用于确定如何把Java对象变成二进制数据，比如，是使用JDK内置的对象序列化，还是将其先序列化为JSON，再把JSON字符串转换为字节数组。

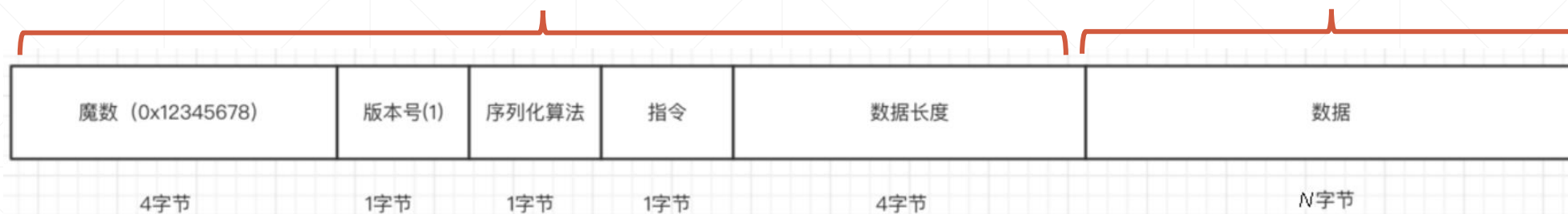


指令可以让服务端知道“要干什么事”，或者“后面收到的数据是什么”，从而可以正确地解析出最后跟着的数据段。比如，HTTP协议中，GET、POST等就是“指令”，与这些指令相配合，需要特定的数据，比如GET指令需要一个URL信息，而POST指令，除了URL，还需要有额外的信息放在HTTP消息的body中。

实现自定义通讯协议

消息头（依据应用场景而自行确定字段，固定长度）

消息体（依据消息头进行读取，长度变化）



依据上述设计方案，使用Netty进行实现，其基本思路是：

- ▶ 所有数据都是放在一个ByteBuf中的，需要先计算出“消息头”的长度，然后，在ByteBuf中收到的数据大于此值时，就可以读取消息头了。
- ▶ 消息头的读取，通常都是调用ByteBuf的readInt()/readBytes()等方法实现的，在实际开发中，通常会为消息头定义一个专有的数据类，创建一个它的实例，从ByteBuf中读取相应内容，赋值给这个对象，然后返回给外界，进行后继处理。

自定义通讯协议中消息头的面向对象封装

```
//自定义通讯协议数据包的基类
@Data
public abstract class Packet {
    //版本
    private Byte version = 1;
    //指令（最多256条，事先约定好每个数字对应什么指令）
    public abstract Byte getCommand();
}
```

```
//这条指令，用于实现用户登录，因此，包容用户名与密码
@Data
public class LoginRequestPacket extends Packet {
    private Integer userId;
    private String userName;
    private String password;

    @Override
    public Byte getCommand() {
        return Command.LOGIN_REQUEST;
    }
}
```

自定义通讯协议，往往需要定义多种指令（称为指令集），并需要为每种指令分配一个唯一的id。同时，相应的指令，往往还需要配套相应的数据，因此，会定义专门的类型，封装这些信息。

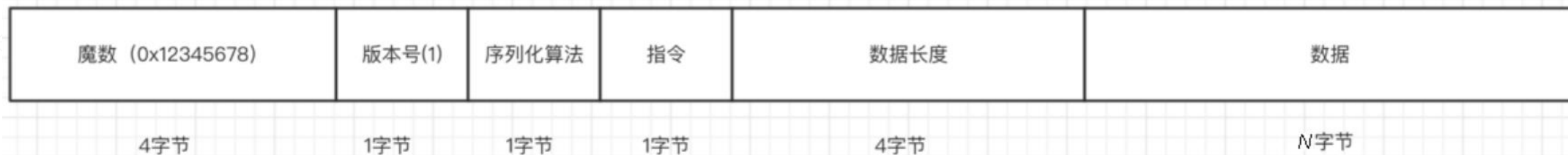
本例中，所有指令都是Packet类型的子类。

```
//在这里，集中定义指令集，可以定义最多256个指令
public interface Command {
    //用于登录的指令id
    Byte LOGIN_REQUEST = 1;

    //依据指令id，获取具体指令实现类型
    static Class<? extends Packet> getRequestType(byte command){
        if(command == LOGIN_REQUEST){
            return LoginRequestPacket.class;
        }
        return null;
    }
}
```

序列化方式的封装

指令和指令相关的数据



```
//如果有多种序列化方式，则给其分配一个唯一的整数
public interface SerializerAlgorithm {
    //值为1时，采用JSON序列化
    byte JSON = 1;
}
```

指令加上相关的数据，构成一个指令对象（即Packet对象），本例将其整个序列化为JSON，再加上“魔数”等元数据，构成一个完整的数据包，就可以写入到ByteBuf中，Netty会把它发送给另一端的计算机。

```
public interface Serializer {
    //默认使用JSON序列化
    byte JSON_SERIALIZER = 1;
    Serializer DEFAULT = new JsonSerializer();

    //获取序列化算法
    byte getSerializerAlgorithm();
    //将Java对象序列化为字节数组
    byte[] serialize(Object object);
    //从字节数组中提取数据，反序列化为Java对象
    <T> T deserialize(Class<T> clazz, byte[] bytes);

    //依据序列化算法id，获取具体的序列化器实例
    static Serializer getSerializer(byte serializerAlgorithm) {
        if(serializerAlgorithm == JSON_SERIALIZER){
            return DEFAULT;
        }
        return null;
    }
}
```


Json序列化的实现代码

```
//使用阿里巴巴的FastJson2进行序列化
public class JSONSerializer implements Serializer {
    //序列化算法名称，采用字节来表示
    @Override
    public byte getSerializerAlgorithm() {
        return SerializerAlgorithm.JSON;
    }

    @Override
    public byte[] serialize(Object object) {
        return JSON.toJSONBytes(object);
    }

    @Override
    public <T> T deserialize(Class<T> clazz, byte[] bytes) {
        return JSON.parseObject(bytes, clazz);
    }
}
```

这个序列化器，主要用于序列化特定的指令对象（包容指令id和指令相关的数据）为JSON，或者从JSON字符串中反序列化指令对象。

将登录指令编码，写入到ByteBuffer

```
public static ByteBuffer encode(Packet packet) {  
    //创建ByteBuffer对象  
    ByteBuffer byteBuf = Unpooled.buffer();  
  
    //按照自定义通信协议的格式，写入相应数据，注意，顺序是重要的  
    byteBuf.writeInt(MAGIC_NUMBER);  
    byteBuf.writeByte(packet.getVersion());  
    byteBuf.writeByte(Serializer.DEFAULT.getSerializerAlgorithm());  
    byteBuf.writeByte(packet.getCommand());  
  
    //将Packet对象先序列化为JSON，再转换为字节数组  
    byte[] bytes = Serializer.DEFAULT.serialize(packet);  
    //写入数据  
    byteBuf.writeInt(bytes.length);  
    byteBuf.writeBytes(bytes);  
  
    return byteBuf;  
}
```

从ByteBuf中读取对象，重建Packet对象

```
public static Packet decode(ByteBuf byteBuf) {  
    //依据自定义协议，计算指令头的总长度  
    //MAGIC_NUMBER(4)+版本号 (1) +序列化算法 (1) +指令Id (1) +数据包长度 (4)  
    int headerLength = 11;  
    if(byteBuf.readableBytes() < headerLength){  
        //读取的数据不够  
        return null;  
    }  
    //跳过MAGIC_NUMBER  
    byteBuf.skipBytes(4);  
    //跳过版本号 (本例用不上)  
    byteBuf.skipBytes(1);  
    //获取序列化算法id  
    byte serializerAlgorithm = byteBuf.readByte();  
    //依据序列化算法id, 获取对应的序列化器实例 (比如, JSON序列化, 则使用JSONSerializer对象)  
    var serializer = Serializer.getSerializer(serializerAlgorithm);  
    //获取指令Id  
    byte command = byteBuf.readByte();  
    //依据指令Id, 获取指令的具体类型 (Class对象)  
    var requestType = Command.getRequestType(command);  
    //获取数据包长度  
    int length = byteBuf.readInt();  
  
    if(byteBuf.readableBytes() < length){  
        //读取的数据不够  
        return null;  
    }  
    //读取指令包容的具体数据  
    byte[] bytes = new byte[length];  
    byteBuf.readBytes(bytes);  
  
    if (requestType != null && serializer != null) {  
        //反序列化, 得到具体的Packet对象  
        return serializer.deserialize(requestType, bytes);  
    }  
    return null;  
}
```

这部分代码，其实做的事情就是读取指令头数据

这里要特别注意，并不能假设一次就能收到所有的数据，因此，在解码时，需要判断一下是否已经收到了足够的数据。

测试编码与解码过程

```
public static void main(String[] args) {  
    //创建用于测试的登录指令  
    LoginRequestPacket loginRequestPacket = new LoginRequestPacket();  
    loginRequestPacket.setUserId(100);  
    loginRequestPacket.setUserName("testUser");  
    loginRequestPacket.setPassword("123456");  
  
    System.out.println("将被序列化的Packet对象");  
    System.out.println(loginRequestPacket);  
    //进行序列化后编码  
    var byteBuf = encode(loginRequestPacket);  
    System.out.println(ByteBufUtil.prettyHexDump(byteBuf));  
  
    //读指针移回开头，以便反序列化  
    byteBuf.readerIndex(0);  
  
    System.out.println("\n反序列化结果");  
    System.out.println(decode(byteBuf));  
}
```

运行截图

```
Run Main x
[C:\Program Files\Java\jdk-21\bin\java.exe" ...
将被序列化的Packet对象
LoginRequestPacket(userId=100, userName=testUser, password=123456)
+-----+
| 0 1 2 3 4 5 6 7 8 9 a b c d e f |
+-----+-----+
|00000000| ab cd ab cd 01 01 01 00 00 00 50 7b 22 63 6f 6d |.....P{"com|
|00000010| 6d 61 6e 64 22 3a 31 2c 22 70 61 73 73 77 6f 72 |mand":1,"passwor|
|00000020| 64 22 3a 22 31 32 33 34 35 36 22 2c 22 75 73 65 |d":"123456","use|
|00000030| 72 49 64 22 3a 31 30 30 2c 22 75 73 65 72 4e 61 |rId":100,"userNa|
|00000040| 6d 65 22 3a 22 74 65 73 74 55 73 65 72 22 2c 22 |me":"testUser","|
|00000050| 76 65 72 73 69 6f 6e 22 3a 31 7d |version":1} |
+-----+-----+

反序列化结果
LoginRequestPacket(userId=100, userName=testUser, password=123456)|

Process finished with exit code 0
```


小结：关于自定义通讯协议的设计



想清楚要设计什么样的“指令集”，通常需要为每个指令分配唯一的标识（通常是一个整数）



设计私有协议，需要仔细考虑要实现的功能，从而得出数据包的结构。



数据包结构如果设计不当，后期再改，问题极多。



数据包结构设计，应该加入到系统分析与设计初期的任务清单之中。

自我训练



“数据包结构设计”，是开发网络应用必须掌握的基础技能，请完成以下典型任务的数据包结构设计：

- (1) 如何设计一个二进制的网络聊天室数据包？
- (2) 如果要实现一个支持断点续传的文件服务器，如何设计数据包结构？